



ΕΡΓΑΣΤΗΡΙΟ 2

Συναρτήσεις

Ορισμός συνάρτησης με την εντολή `def`

Τα περισσότερα προγράμματα τα οποία φτιάξαμε μέχρι τώρα θα μπορούσαμε να τα δούμε ως ολοκληρωμένες μεθόδους οι οποίες αποτελούνται από τα εξής μέρη

1. εισάγονται δεδομένα,
2. γίνονται υπολογισμοί και
3. εξάγεται και τυπώνεται ένα αποτέλεσμα.

Για παράδειγμα, μπορούμε να βρούμε τον μεγαλύτερο μεταξύ δύο αριθμών ως εξής

```
x = 5.0
y = 6.0

if x > y:
    m = x
else:
    m = y

print(m)
```

Οργάνωση του προγράμματος

Για έναν προγραμματιστή είναι πολύ σημαντικό να οργανώσει το πρόγραμμά σου σε τμήματα (blocks) έτσι ώστε να γίνεται εμφανής η δομή του. Με αυτόν τον τρόπο ο προγραμματιστής θα έχει πλεονεκτήματα.

- Θα μπορεί εύκολα να καταλάβει το πρόγραμμά του ακόμα και αν το δει μετά από πολύ καιρό
- Θα μπορεί να δώσει το πρόγραμμά του για χρήση σε άλλον προγραμματιστή.
- Θα έχει τη δυνατότητα να επαναχρησιμοποιήσει κάποια σημαντικά μέρη του κώδικά του πολλές φορές μέσα στο ίδιο πρόγραμμα.

Μία πρώτη συνάρτηση.

Ας διαχωρίσουμε το παραπάνω πρόγραμμα σε τμήματα. Το τμήμα του προγράμματος το οποίο κάνει τον υπολογισμό του μεγίστου μπορούμε να το διαχωρίσουμε ως εξής

```
def maximum(x, y):
    if x > y:
        m = x
    else:
        m = y
    return m
```



Η λέξη `def` είναι μία εντολή της python και ορίζει μία συνάρτηση. Έχουμε τους εξής κανόνες για τον ορισμό της συνάρτησης.

- Η πρώτη γραμμή περιέχει το όνομα της συνάρτησης (η παραπάνω λέγεται `maximum`). Το όνομα μιας συνάρτησης ακολουθεί τους συνηθισμένους κανόνες της Python για τα ονόματα μεταβλητών.
- Ορισμένες μεταβλητές (ορίσματα) βρίσκονται σε παρένθεση μετά το όνομα της συνάρτησης και αυτές είναι τα δεδομένα τα οποία είναι απαραίτητα για να εκτελεστούν οι εντολές μέσα στην συνάρτηση. Η πρώτη αυτή γραμμή τελειώνει με την άνω-κάτω τελεία.
- Όλες τις εντολές που περιέχονται στη συνάρτηση βρίσκονται σε εσοχή (tab).
- Η συνάρτηση τελειώνει με την εντολή `return` η οποία καθορίζει ποια είναι η τιμή που επιστρέφει η συνάρτηση.

Βλέπουμε λοιπόν ότι μία γενική μορφή για την συνάρτηση είναι η εξής.

```
def functionName(variable1,variable2,...):  
    statements  
    return variable
```

Ένα ολοκληρωμένο πρόγραμμα με συνάρτηση

Εφόσον έχουμε γράψει την παραπάνω συνάρτηση `maximum` το πρόγραμμα η οποία βρίσκει τον μέγιστο μεταξύ δύο αριθμών μπορεί να την καλέσουμε ως εξής:

```
a = 5.0  
b = 6.0  
result = maximum(a,b)  
print(result)
```

Οι παραπάνω εντολές αποτελούν το *κυρίως πρόγραμμα*. Το ολοκληρωμένο πρόγραμμα αποτελείται τώρα από δύο τμήματα: την συνάρτηση και το κυρίως πρόγραμμα (το οποίο συνήθως ακολουθεί τη συνάρτηση)

Ας δούμε σε λεπτομέρεια τι ακριβώς συμβαίνει στο παραπάνω πρόγραμμά μας στο οποίο περιέχεται η δήλωση της συνάρτησης (`maximum(x,y)` με παραμέτρους εισόδου τα `x,y`) και στο κυρίως πρόγραμμα καλούμε τη συνάρτηση με το όνομά της `maximum(a,b)`.

- Λέμε ότι η συνάρτηση `maximum` καλείται με τις πραγματικές παραμέτρους (ή ορίσματα) `a,b`. Αυτές μπορεί να είναι μεταβλητές οι οποίες έχουν τιμές π.χ.,
`a=5.0`
`b=6.0`
`m=maximum(a,b)`
αλλά μπορεί να είναι και αριθμοί π.χ.,
`m=maximum(5.0,6.0)`
- Με την κλήση της συνάρτησης συμβαίνουν δύο πράγματα: Οι παράμετροι εισόδου της συνάρτησης (`x` και `y`) παίρνουν τις τιμές `x=5` και `y=6` αντιστοίχως από τις μεταβλητές `a` και `b` του κυρίως προγράμματος) και το πρόγραμμα συνεχίζει εκτελώντας τις εντολές εντός της συνάρτησης.



- Όταν βρεθεί εντολή `return` ο έλεγχος επιστρέφει στο κυρίως πρόγραμμα.
- Η τιμή της μεταβλητής που βρίσκεται δίπλα στην `return` (δηλαδή `m`) είναι η επιστρεφόμενη τιμή της συνάρτησης. Αυτή δίδεται στο κύριο πρόγραμμα (στη μεταβλητή `result`) αριστερά της ισότητας στην κλήση της συνάρτησης.
- Ο έλεγχος έχει τώρα επιστρέψει στο κυρίως πρόγραμμα και εκτελούνται οι εντολές μετά της κλήση της συνάρτησης.
- Η τελευταία εντολή του κυρίως προγράμματος τυπώνει το αποτέλεσμα που πήραμε από την εκτέλεση της συνάρτησης.

Παράδειγμα 1

Γράψτε μία συνάρτηση η οποία να υπολογίζει την τιμή του πολυωνύμου $f(x) = x^2 - 2x + 1$. Καλέστε την από το κυρίως πρόγραμμα για μία τιμή του x και τυπώστε τα x και $f(x)$.

```
def f(x):          # εδώ είναι η συνάρτηση f(x)
    fvalue = x**2 - 2*x + 1
    return fvalue

x = 3.0           # εδώ είναι το κυρίως πρόγραμμα
result = f(x)
print(x, result)
```

Παράδειγμα 2

Γράψτε μία συνάρτηση η οποία να ελέγχει αν το όνομά που δίνεται σαν είσοδος τελειώνει σε "ακης" (akis). Σε αυτή την περίπτωση η επιστρεφόμενη τιμή θα είναι `True`, αλλιώς να επιστρέφεται `False`.

```
def nameEnd(s):
    if s[-4:] == 'akis':
        return True
    else:
        return False

name = input("Give a name: ")
print(nameEnd(name))
```

Εντολή `return` και απουσία της

Σημειώστε ότι η εμφάνιση της εντολής `return` σε οποιοδήποτε σημείο στο σώμα μιας συνάρτησης τερματίζει την εκτέλεση των εντολών της συνάρτησης και επιστρέφει τη ροή του προγράμματος στο σημείο αμέσως μετά την κλήση της. Η χρήση της εντολής `return` είναι προαιρετική. Αν αυτή δεν εμφανίζεται ή η εμφάνισή της δεν ακολουθείται από κάποια έκφραση, τότε η συνάρτηση επιστρέφει την τιμή `None`.

Παράδειγμα

Ας ορίσουμε μία συνάρτηση η οποία τυπώνει το μήνυμα `Hello!` όποτε κληθεί:

```
def sayHello():
    print('Hello!')
```



Ας την καλέσουμε

```
sayHello()
```

θα πάρουμε το αποτέλεσμα:

```
Hello!
```

Αν γράψουμε

```
result = sayHello()
```

τότε η μεταβλητή `result` παίρνει την τιμή `None` διότι η συνάρτηση δεν επιστρέφει τίποτα (δεν υπάρχει η εντολή `return` άλλωστε μέσα στη συνάρτηση. Μπορείτε να εξακριβώσετε το πιο πάνω γράφοντας:

```
print(result)
```

Μπορείτε να παρατηρήσετε ότι η συνάρτηση αυτή δεν έχει παραμέτρους εισόδου και στην κλήση της συνάρτησης πρέπει να μουν οι παρενθέσεις αλλά μέσα δε θα δοθεί κάτι.

Παρατήρηση

Το κεντρικό πλεονέκτημα των συναρτήσεων είναι ότι μπορούμε να τις καλούμε επανειλημμένα από το κυρίως πρόγραμμα. Ο κώδικας που περιέχεται στη συνάρτηση εκτελείται σε κάθε κλήση της από το κυρίως πρόγραμμα.

Modules

Ένα `module` είναι ένα αρχείο που περιέχει ορισμούς συναρτήσεων που χρησιμεύουν για την επίλυση ενός συγκεκριμένου προβλήματος. Η Python κατά την εγκατάστασή της στον υπολογιστή αποθηκεύει στο δίσκο ένα φάκελο (folder) ένα μεγάλο αριθμό από `modules`, το καθένα με τα δικά του χαρακτηριστικές συναρτήσεις. Το όνομα του αρχείου καθορίζει και το όνομα του `module` και έχει επέκταση όπως και τα προγράμματα σε Python τα γράμματα `py` πχ `math.py` που αφορά σε μαθηματικές συναρτήσεις. Εκτός των αυτόματα εγκατεστημένων `modules` μπορούμε να γράψουμε τα δικά μας.

Α' τρόπος ενσωμάτωσης ενός module στο πρόγραμμά μας

Η ενσωμάτωση ενός `module` στο πρόγραμμα επιτυγχάνεται μέσω της εντολής `import` στην αρχή το προγράμματος. Για την ενσωμάτωση και χρήση μαθηματικών συναρτήσεων χρειαζόμαστε το `math.py` που επιτυγχάνεται με την εντολή:

```
import math
```

Η χρήση των συναρτήσεων που υπάρχουν στο `module` γίνεται με την αναφορά στο `module` και στην επιθυμητή συνάρτηση.



Παράδειγμα Οι συναρτήσεις του math καλούνται ως εξής:

math.συνάρτηση(παράμετροι) π.χ.

```
import math
y=math.cos(x) # υπολογίζει το συνημίτονο του x και αναθέτει το
               # αποτέλεσμα στη μεταβλητή y
print(math.pi) # τυπώνει τον αριθμο π=3.14159
```

Β' τρόπος ενσωμάτωσης ενός module στο πρόγραμμά μας

Υπάρχει ακόμα ένας τρόπος για να ενσωματώσουμε και να καλέσουμε μια συνάρτηση του math module όπως φαίνεται πιο κάτω:

```
from math import cos,pi
y=cos(x) # υπολογίζει το συνημίτονο του x και αναθέτει το
          # αποτέλεσμα στη μεταβλητή y
print(pi) # τυπώνει τον αριθμο π=3.14159
```

Με τον τρόπο αυτό ενσωματώνουμε μόνο τις συναρτήσεις που θέλουμε από το math module και όταν καλούμε μια συνάρτηση δεν πρέπει να βάζουμε το math μπροστά.

Modules οριζόμενα από τον χρήστη

Ένα module το οποίο ορίζεται από τον χρήστη είναι ένα αρχείο το οποίο έχει ως επέκταση py (πχ. synartiseis.py) και περιέχει ένα σύνολο συναρτήσεων ή κλάσεων που μπορούν να χρησιμοποιηθούν για την επίλυση ενός συγκεκριμένου προβλήματος. Το αρχείο αυτό πρέπει να είναι αποθηκευμένο στον φάκελο (folder) που είναι αποθηκευμένο το κυρίως πρόγραμμα του χρήστη.

Η ενσωμάτωση ενός module στο πρόγραμμα επιτυγχάνεται μέσω της εντολής import στην αρχή το προγράμματος. Για την ενσωμάτωση στο κύριο πρόγραμμα και χρήση των συναρτήσεων που περιέχονται στο αντίστοιχο module πρέπει να γίνεται εισαγωγή στο κύριο πρόγραμμα. Αυτό επιτυγχάνεται με την εντολή import π.χ.

```
import synartiseis
```

Η χρήση κάποια συνάρτησης του module επιτυγχάνεται με την αναφορά στο module και στην αντίστοιχη συνάρτηση. Π.χ Έστω ένα module με ονομα synartiseis για τον υπολογισμό διαφόρων μεταβλητών στη μισθοδοσία υπαλλήλων. Έστω επίσης ότι περιέχει εκτός των άλλων μία συνάρτηση με όνομα ypologismos_forou που δέχεται ως παράμετρο εισόδου τις φορολογητέες αποδοχές του υπαλλήλου. Για να λάβουμε τον αντίστοιχο φόρο σε μία μεταβλητή με το όνομα foros η αναφορά στην συνάρτηση θα γίνει με την εξής εντολή:

```
import synartiseis
foros=synartiseis.ypologismos_forou(apodoxes)
```

Η πιο πάνω ενσωμάτωση και κλήση της συνάρτησης μπορεί να γίνει και με τον Β' τρόπο ως ακολούθως:



```
from synartiseis import ypologismos_forou  
foros=ypologismos_forou(apodoxes)
```

Εργαστηριακές Ασκήσεις

1. Δίνεται η παρακάτω συνάρτηση

```
def myFunction(x) :  
    total = 0  
    for i in range(x) :  
        total += i * (i-1)  
    return total
```

Τοποθετήστε τη συνάρτηση σε ένα αρχείο `myfunctions.py` και αποθηκεύστε. Στον ίδιο φάκελο (directory), δημιουργήστε το πρόγραμμά σας σε ένα αρχείο `myprogram.py`. Το πρόγραμμά σας θα πρέπει να κάνει `import` τη συνάρτηση `myFunction` για να τη χρησιμοποιήσει (χρησιμοποιώντας τον Β' τρόπο). Γράψτε στο πρόγραμμα τις κατάλληλες εντολές, ώστε να καλεί την παραπάνω συνάρτηση με τιμή εισόδου το 5 και να εκτυπώνει το αποτέλεσμα που επιστρέφει η συνάρτηση. Σιγουρευτείτε πως κατανοείτε γιατί η συνάρτηση επιστρέφει αυτήν την τιμή για $x=5$.

2. Γράψτε μια συνάρτηση `max3(x, y, z)` στο αρχείο `myfunctions.py` που να δέχεται 3 αριθμούς και να επιστρέφει το μεγαλύτερο. Μετά ενσωματώστε και καλέστε τη συνάρτηση `max3` από το αρχείο `myprogram.py` ως εξής `print(max3(4, 6, 2))` και θα πρέπει να εκτυπώνει «6».
3. Ένα δίσεκτο έτος κατά το Γρηγοριανό ημερολόγιο είναι μια χρονολογία η οποία ικανοποιεί ένα από τα πιο κάτω κριτήρια

- a. διαιρείται με το 4 αλλά όχι από 100
- b. διαιρείται με το 400

Για παράδειγμα, το 1896, το 1904, και το 2000 είναι δίσεκτες χρονιές, αλλά το 1900 και το 2100 δεν είναι. Γράψτε μια συνάρτηση `leap_year` στο αρχείο `myfunctions.py` που παίρνει σαν παράμετρο ένα ακέραιο (το έτος) και επιστρέφει `True` εάν είναι ένα δίσεκτο έτος και `False` αν δεν είναι. Καλέστε τη συνάρτηση αυτή 2 φορές από το αρχείο `myprogram.py` την πρώτη φορά με είσοδο το έτος 1996 και τη δεύτερη φορά το 2100 και τυπώστε αυτό που επιστρέφεται.

4. Γράψτε μια συνάρτηση `countCharInString` στο αρχείο `myfunctions.py` που να παίρνει σαν είσοδο ένα `string` και ένα χαρακτήρα και επιστρέφει πόσες φορές υπάρχει ο χαρακτήρας μέσα στο `string`. Στο αρχείο `myprogram.py` ζητείστε από το χρήστη να δώσει ένα `string` και ένα χαρακτήρα και μετά καλέστε τη συνάρτηση `countCharInString` δίνοντας σαν είσοδο τα δεδομένα που έδωσε ο χρήστης και τυπώστε αυτό που επιστρέφει.

Παραδείγματα Εκτέλεσης (με πράσινο η είσοδος του χρήστη)

Give a string: **symvoloseira**

Give a char you want: **o**

The character o appears 2 times in the string symvoloseira



5. Γράψετε μια συνάρτηση `printEvenBetween` στο αρχείο `myfunctions.py` που να παίρνει σαν είσοδο δύο ακέραιους `A` και `B` και να τυπώνει τους ζυγούς αριθμούς μεταξύ του `A` και του `B` (συμπεριλαμβανομένων). Το `A` πρέπει να είναι πιο μικρό από το `B`. Αν δεν ισχύει αυτό (δηλαδή αν $A > B$) τότε η συνάρτηση πρώτα να ανταλλάζει τις τιμές των δύο μεταβλητών και μετά να τυπώνει τους ζυγούς. Στο αρχείο `myprogram.py` ζητείστε από το χρήστη να δώσει 2 ακέραιους αριθμούς και μετά καλέστε τη συνάρτηση `printEvenBetween` δίνοντας σαν είσοδο τα δεδομένα που έδωσε ο χρήστης.

Παραδείγματα Εκτέλεσης (με πράσινο η είσοδος του χρήστη)

Give first number: 21

Give second number: 45

22 24 26 28 30 32 34 36 38 40 42 44